

Modeling and Optimizing a Battery Charging Profile

In this project, you will use MATLAB to model a battery charging profile. You will fit real lithium-ion battery data in MATLAB, compute energy transfer through numerical integration, and analyze key performance characteristics such as charge-rate behavior and efficiency losses.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this [video](#).

Suggested Tasks

1. Given data for a lithium-ion battery charging profile, use MATLAB to fit the equation for voltage provided in the Project Description.
2. Plot the following curves: voltage vs. time, current vs. time, and power vs. time.
3. Use integration to compute the total energy delivered to the battery (area under power-time curve).
4. Analyze the rate of voltage change at different intervals (calculate derivatives), compute the time to reach 80% and 100% charge, and compute the energy lost due to resistance.

Table of Contents

Break Down the Problem	1
Task 1: Fit the voltage equation to a lithium-ion battery profile	2
Task 2: Plot the electrical terms that express how energy moves in a circuit	7
Task 3: Compute analytical results	10
Interpretation of Results	14

Break Down the Problem

In the space below, define the problem statement:

Lithium-Ion battery charging involves non-linear changes in current and voltage over time, leading to difficulties in predicting the battery's charge time and energy efficiency.

List the requirements, constraints, and success criteria for the project:

The requirements of this project are to use the MATLAB Curve Fitting Toolbox and the provided dataset to fit the equation onto the Lithium-Ion battery's charging profile for analysis.

One of the constraints identified within the project is the use of a simple RC model equation to represent the battery's behavior. This is a constraint because the RC model equation does not account for circumstances that may be contributing to the Lithium-Ion battery's non-linearity (e.g. temperature, chemical behavior, etc.).

For this project to be considered successful, the battery data must be imported and fitted correctly to ensure accuracy of the results when analyzing energy efficiency and charging time. Another criteria for success is finding the values necessary for battery charging profile analysis (e.g. current vs. time, voltage vs. time, energy delivered, resistive energy loss, charging time, etc.).

What is your proposed solution?

Using the provided tools in MATLAB and the battery dataset, the proposed solution is to use the capacitor charging equation to estimate a Lithium-Ion battery's charging time and energy efficiency in order to determine areas for improvement in the battery's performance.

List the steps you will need to take to achieve your proposed solution:

First, we will load and organize the battery charging data. Next, we will analyze the voltage, current, and time data and generate plots to better understand the battery's charging behavior. We will then use mathematical and numerical methods to model the charging profile, calculate important charging characteristics, and estimate energy delivered and energy lost due to resistance. Finally, we will summarize and interpret the results to evaluate the battery's charging performance.

What challenges do you face in the process of achieving the solution?

Some possible challenges include identifying important trends and charging stages from noisy experimental data, selecting appropriate mathematical methods to analyze the battery's behavior, and determining how accurately the model represents the actual charging data.

Task 1: Fit the voltage equation to a lithium-ion battery profile

Given real time-series voltage data for a lithium-ion battery, use MATLAB Curve Fitting Toolbox to fit the

equation $V(t) = V_{\max} \left(1 - e^{-\frac{t}{RC}} \right)$ and model the battery charging profile as a simple RC-circuit analog.

Load MathWorks example lithium-ion batteries data by following the steps [here](#). The page also includes information about the dataset, how the data was recorded, and the battery's maximum voltage.

Note: The battery from this example dataset has been cycled to failure (charged and discharged repeatedly). For this project, you may choose to use data from a single cycle and focus only on the period of time within a cycle that the battery is being charged. The battery is fully charged at 3.6V. This example battery uses a dynamic fast charging profile, but for this project, you will attempt to model the charging profile using a simple exponential capacitor charging equation.

Then, follow the workflow outlined in the [documentation](#) to fit the voltage equation to your selected data. Plot the data and fitted curve in the same figure and, finally, print the goodness-of-fit statistics.

Approach and Methodology

The main objective of task 1 was to make a fit using measured lithium-ion battery data towards a simplified RC charging equation: $V(t) = V_{\max} \left(1 - e^{-\frac{t}{RC}} \right)$. Next, the project loaded and prepared using the battery dataset from Mathworks to select and process cycle 1. Its timestamps were later converted into elapsed seconds starting from $t = 0$.

The Curve Fitting Toolbox is then used to define a custom equation based on the RC charging equation in order to estimate , generate the fitted curve, and calculate the goodness-of-fit statistics. The measured voltage and fitted RC curve were also plotted together for comparison

Results and Analysis

The fitted curve demonstrated the overall charging behavior of the battery from the dataset. It followed the expected behavior of a RC charging model, where the voltage rapidly and constantly increased at the beginning of charging and then eventually reached its maximum voltage. However, the fitted curve did not show every changes or fluctuation in the voltage data/curve since the given RC equation is a simplified model compared with a complicated lithium-ion battery that is also affected by external or internal factors. Also, the goodness-of-fit and plotted curve were used to evaluate how well the given equation representing real battery behavior.

Overall, this fitted curve using the given simplified RC equation is useful for demonstrating the general charging pattern, but it is not good to fit and analyze with complicated data.

Challenges encountered

The only challenge for task 1 was to customize an equation based on the given RC model using MATLAB and fitting it to the real lithium-ion battery data, where it was difficult to perfectly match them since the actual voltage data have different variations and changes that can not be demonstrated with a simple RC equation.

Design Decision and Tradeoff:

A simple RC charging equation is easy to understand and analyze making it easier to fit it to the battery data. However, although it can represent the main charging trend, it does not work well under all the complications and variations of data from a real-world scenario.

```
% Insert your code in the spaces below (or make use of helper functions and
indicate where
% they can be found). Ensure your code is well-documented.
```

```
% STEP 1
% Load the example battery dataset
dataFile =
matlab.internal.examples.downloadSupportFile("predmaint","batteryagingdata/
singlecell/v1/singleCellLifeTimeData.zip");
unzip(dataFile)
load("singleCellLifeTimeData.mat")
head(data,5)
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current
1	0	13-May-2017 03:21:40	-1780	0	0	0
2	0.0001	13-May-2017 03:21:40	-1780	0	0	0
3	9.9983	13-May-2017 03:21:50	-1770	0	0	0
4	20.002	13-May-2017 03:22:00	-1760	0	0	0
5	30.001	13-May-2017 03:22:10	-1750	0	0	0

```
% STEP 2
% Pre-process the data to focus on a single cycle, and the period of time
% within that cycle that the battery was being charged
```

```
cycleData = data(data.Cycle_Index == 1, :)
```

```
cycleData = 1078x15 table
```

...

	Data_Point	Test_Time	DateTime	Step_Time	Step_Index
1	2834	1.0168e+04	14-May-2017 03:42:34	0.0123	12
2	2835	1.0168e+04	14-May-2017 03:42:34	0.0533	5
3	2836	1.0168e+04	14-May-2017 03:42:34	0.0798	5
4	2837	1.0168e+04	14-May-2017 03:42:35	0.0949	5
5	2838	1.0168e+04	14-May-2017 03:42:35	0.1087	5
6	2839	1.0168e+04	14-May-2017 03:42:35	0.1236	5
7	2840	1.0168e+04	14-May-2017 03:42:35	0.1374	5
8	2841	1.0168e+04	14-May-2017 03:42:35	0.1521	5
9	2842	1.0168e+04	14-May-2017 03:42:35	0.1653	5
10	2843	1.0169e+04	14-May-2017 03:42:35	0.1791	5
11	2844	1.0169e+04	14-May-2017 03:42:35	0.1957	5
12	2845	1.0169e+04	14-May-2017 03:42:35	0.2105	5
13	2846	1.0169e+04	14-May-2017 03:42:35	0.2246	5
14	2847	1.0169e+04	14-May-2017 03:42:35	0.2375	5

⋮

```
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1))
```

```
cycleData = 1078x15 table
```

...

	Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index
1	2834	1.0168e+04	0	0.0123	12	1
2	2835	1.0168e+04	0.1578	0.0533	5	1
3	2836	1.0168e+04	0.1843	0.0798	5	1
4	2837	1.0168e+04	0.1994	0.0949	5	1
5	2838	1.0168e+04	0.2132	0.1087	5	1
6	2839	1.0168e+04	0.2281	0.1236	5	1
7	2840	1.0168e+04	0.2419	0.1374	5	1
8	2841	1.0168e+04	0.2566	0.1521	5	1
9	2842	1.0168e+04	0.2698	0.1653	5	1
10	2843	1.0169e+04	0.2836	0.1791	5	1
11	2844	1.0169e+04	0.3002	0.1957	5	1
12	2845	1.0169e+04	0.3150	0.2105	5	1

	Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index
13	2846	1.0169e+04	0.3291	0.2246	5	1
14	2847	1.0169e+04	0.3420	0.2375	5	1

⋮

```
plot(cycleData.DateTime,cycleData.Voltage, '-r', 'LineWidth', 1.5)
ylabel('Voltage (V)')
xlabel("Time (s)")
title('Measurements over one cycle')
grid on %Add grid to the graph
```



```
% STEP 3
% Follow the provided workflow to fit the voltage equation to your selected
% data.
% Hint: Vmax is known, so the only unknown in the equation is RC (also
% known as tau, the RC time constant – it is equivalent to the time it takes
% for a capacitor to reach ~63.2% of its max voltage). When you are fitting
% the curve to the data, you are essentially estimating the unknown
% parameter, tau.
```

```
targetVoltage = 0.632 * max(cycleData.Voltage); % 2.2752 V
```

```
% Find the time at which voltage is closest to 2.2752 V
[~, idx] = min(abs(cycleData.Voltage - targetVoltage));
tau0 = cycleData.DateTime(idx);
```

```
battChargeEq = fittype( ...
    'Vmax*(1-exp(-t/tau))', ...
```

```

    'independent', 't', ...
    'dependent', 'v', ...
    'coefficients', {'tau'}, ...
    'problem', {'Vmax'});

[battFit, gof] = fit( ... %Add gof to both return the fitted model and
goodness-of-fit statistics.
    cycleData.DateTime, ...
    cycleData.Voltage, ...
    battChargeEq, ...
    'StartPoint', tau0, ...
    'Lower', eps, ...
    'problem', max(cycleData.Voltage));

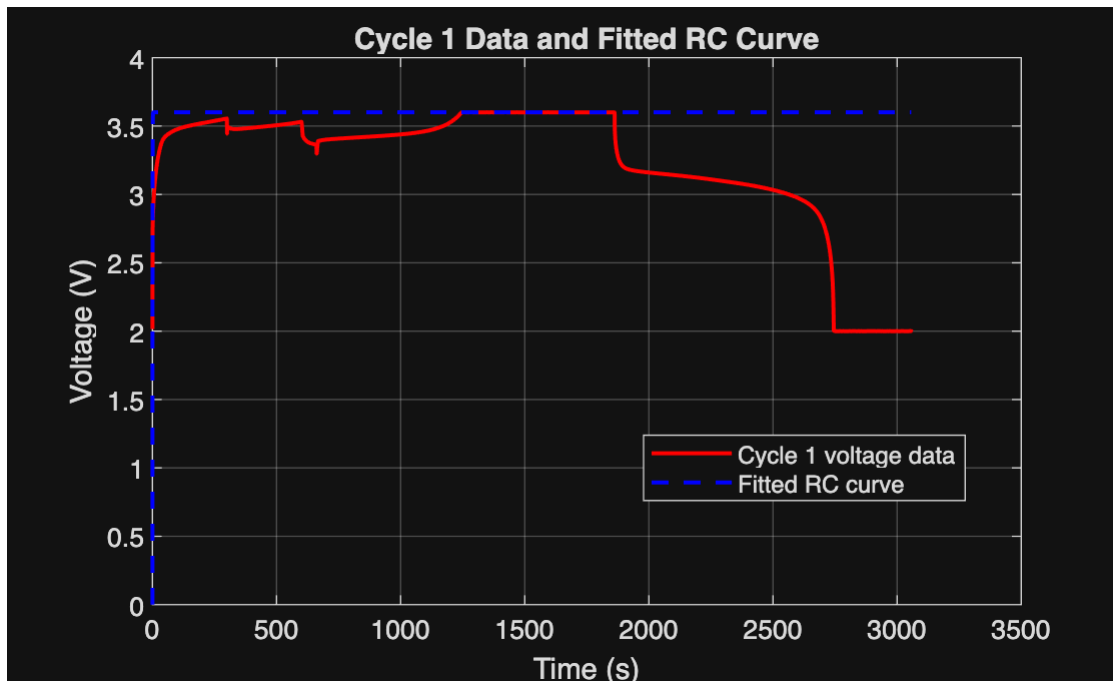
% STEP 4
% Plot the data and the fitted curve in the same figure.

figure % Open a new figure graph, avoiding overwriting graph from step 2

% Plot fitted curve in blue and measured data in red.
plotLines = plot(battFit, 'b--', ...
    cycleData.DateTime, cycleData.Voltage, 'r-');
% Set both lines to the same width.
set(plotLines, 'LineWidth', 1.5);

% Label the graph.
xlabel('Time (s)')
ylabel('Voltage (V)')
title('Cycle 1 Data and Fitted RC Curve')
legend('Cycle 1 voltage data', 'Fitted RC curve', 'Location', 'best')
grid on

```



```
% STEP 5
% Display the goodness-of-fit statistics.
fprintf('\nGoodness-of-Fit Statistics:\n')
```

Goodness-of-Fit Statistics:

```
fprintf('SSE: %.6f\n',gof.sse) % Total squared error
```

SSE: 430.822829

```
fprintf('R-squared: %.6f\n',gof.rsquare) % Variation explained
by the model
```

R-squared: -0.653842

```
fprintf('Adjusted R-squared: %.6f\n',gof.adjrsquare) % R-squared adjusted
for model size
```

Adjusted R-squared: -0.653842

```
fprintf('RMSE: %.6f V\n',gof.rmse) % Root mean squared
error
```

RMSE: 0.632472 V

```
fprintf('Degrees of freedom: %d\n',gof.dfe) % Data points minus
fitted parameters
```

Degrees of freedom: 1077

Task 2: Plot the electrical terms that express how energy moves in a circuit

To visualize the battery's charging policy, write code to visualize the current, voltage, and power measurements for one charging cycle. Present the visualization as one figure with three subplots, one each for:

- Voltage vs. time
- Current vs. time
- Power vs. time

Approach and methodology:

From task 1, we started by using the data and the time that elapsed from 1 cycle of charge. While voltage and current used directly from the table, power had to be computed as $P(t) = V(t) \times I(t)$. Voltage, current, and power were made into 3 subplots against time. Each subplot has its own axis labels and grid lines so the different variables could be compared directly.

Results and analysis:

The voltage level starts around 3.3 V and rises through its charging phase, plateauing near 3.5–3.6 V at around 1200–1850 s. This is consistent with a standard lithium-ion charging profile. The battery's current stays high early but tapers as it approaches its voltage limit, which is when the voltage steadies out. When step-like dips occurred in voltage, the current's dips also coincided exactly around 300–650 s. This represents the battery's resistance, where change in current results in an immediate voltage drop ($V = I \times R$). Around 1850 s, the battery cycle transitions to a steady discharge where the current is about -4.5 A. The voltage decreases toward the 2.0 V cutoff at around 2750 s. The power subplot's shape is similar to current's subplot, which peaks near 22 W at the start and dips to about -14 W during discharge. The voltage stays stable compared to the other subplots with each phase.

Challenges encountered:

There were some errors in the code like case sensitivity and that resulted in the response "unable to resolve the name" error that made me have to back track to fix. A variable I had a hard time with was "cycleData", which also caused problems when I used "cycleData.DateTime". It was a variable I had to reassign to make the section run without any errors. Another problem was running live script sections individually instead of running all the sections from top-to-bottom. Running all the sections caused variables from earlier sections to be undefined when a later section ran. Trying to compute power from the voltage and current datasets was also a challenge. Knowing to use `(.*)` instead of `(*)` was something we had to figure out.

Design decisions and trade-offs:

Since voltage, current, and power have different scales we used three subplots instead of one combined plot. If we used one combined plot for all those variables, it would be hard to read since they would be on the same axis. The tradeoff was 3 short graphs that were still hard to read. I had to add some extra code so that it could be read more easily.

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where they can be found). Ensure your code is well-documented.

% STEP 1
% Create a figure handle and initialize the first subplot. Plot voltage vs.
% time.
```

```

S = load("singleCellLifeTimeData.mat");
cycleData = S.data(S.data.Cycle_Index == 1, :);
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1));

figure('Position', [100, 100, 700, 800])
subplot(3, 1, 1);
plot(cycleData.DateTime, cycleData.Voltage, '-r', 'LineWidth', 1.5)
xlabel('Time (s)');
ylabel('Voltage (V)');
title('Voltage vs. Time');
grid on

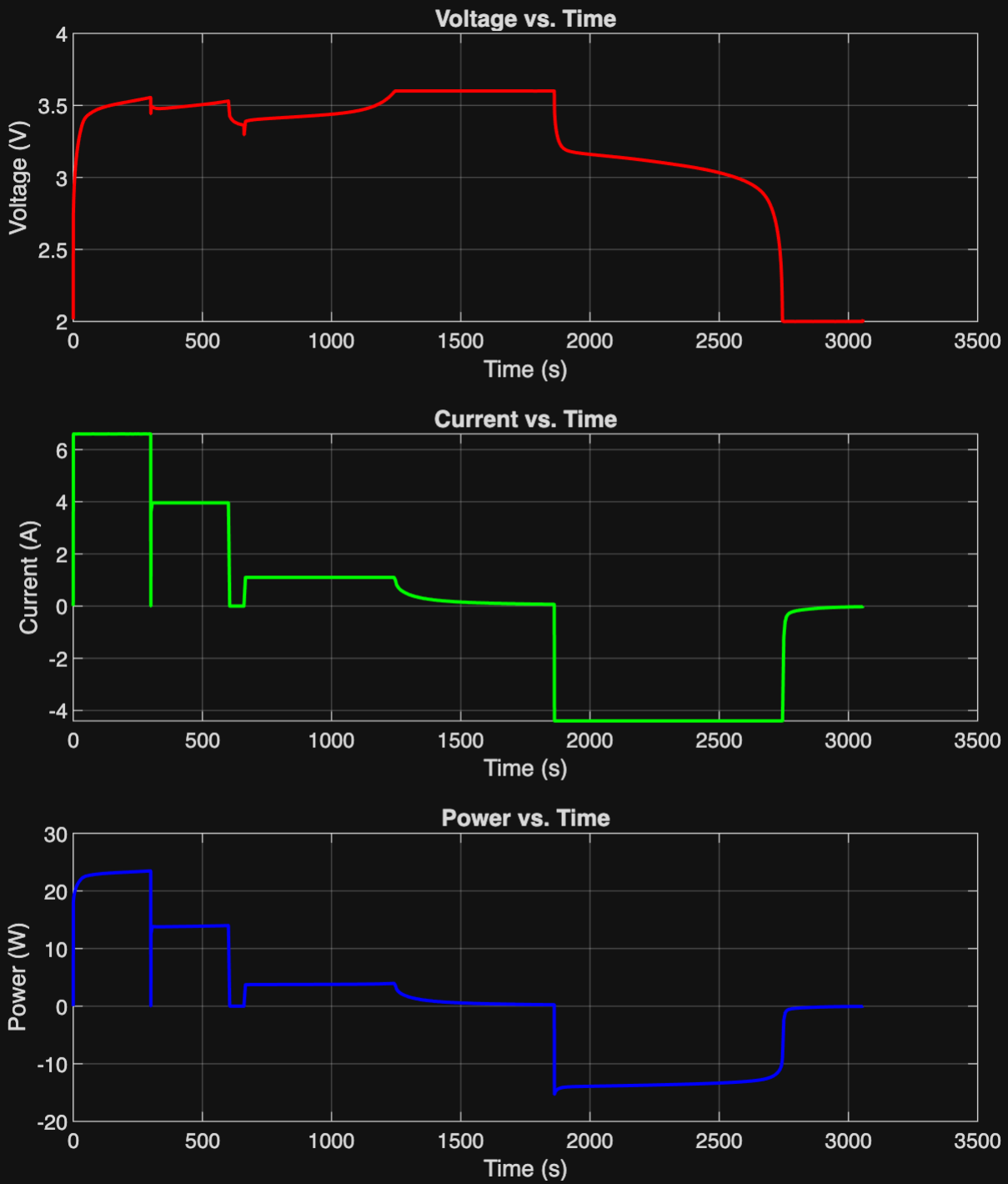
% STEP 2
% Use a second subplot to plot current vs. time.

subplot(3, 1, 2);
plot(cycleData.DateTime, cycleData.Current, 'g-', 'LineWidth', 1.5);
xlabel('Time (s)');
ylabel('Current (A)');
title('Current vs. Time');
grid on

% STEP 3
% Use a third subplot to plot power vs time.
% Hint:  $P = I * V$  where I is current in amps. You will first
% need to compute power before you can create this plot.

cycleData.Power = cycleData.Current .* cycleData.Voltage;
subplot(3, 1, 3);
plot(cycleData.DateTime, cycleData.Power, 'b-', 'LineWidth', 1.5);
xlabel('Time (s)');
ylabel('Power (W)');
title('Power vs. Time');
grid on

```



Task 3: Compute analytical results

To get started on using MATLAB for numerical integration and differentiation, start with this [doc page](#).

- Conduct a rate-of-change analysis by computing the derivative at key intervals in the voltage data
- Compute the time required to reach 80% and 100% charge
- Compute the total energy delivered to the battery by using integration to calculate the area under the power-time curve.
- Estimate the resistive energy loss
- Save the results to a summary table.

Approach and Methodology:

From Task 1, we created vectors containing the time, voltage, and current data from the cycle that we analyzed. Next, using the graphs generated in Task 2 for voltage vs. time, current vs. time, and power vs. time, we identified key intervals to use in our analysis. We used the gradient function to estimate the rate of voltage change, numerical integration to calculate cumulative charge and energy delivered, and Joule's heating formula to estimate resistive energy loss.

Results and Analysis:

The results are listed in the summary table. The rates of voltage change for the first three key intervals were positive, which is expected during the charging cycle. The only exception was the interval from 1300-1800 seconds, which had a slightly negative rate that was nearly zero. This matches the graph because the voltage has a nearly flat slope during that interval. The battery reached 80% of its recorded cumulative charge at approximately 596.6 seconds, while the full charging interval lasted approximately 1846.77 seconds. Most of the charge was delivered early because the charging current was much higher during the first two intervals. The total energy delivered during charging was approximately 13,679 J, while the estimated resistive energy loss was approximately 326.37 J.

Challenges encountered:

One challenge that we encountered during this task was determining appropriate key intervals and excluding excess noise. When analyzing the graphs, we noticed significant changes around 0-300 seconds, 300-600 seconds, 600-1250 seconds, and 1250-1850 seconds, which led us to use these intervals in our analysis. The main issue was noise near the edges of the intervals, where abrupt jumps and drops occurred. To address this, we excluded approximately 50-100 seconds from the ends of each interval depending on the amount of noise present. Another challenge was determining when the battery reached 80% charge. Identifying the end of charging was relatively straightforward because the current and power became negative when the battery began discharging. To estimate the 80% point, we calculated the cumulative charge, treated its final value as the 100% reference, calculated 80% of that value, and found the first index where the cumulative charge reached or exceeded the target.

Design decisions and trade-offs:

The key intervals were selected by excluding approximately 50-100 seconds from the ends of each estimated charging stage. This decision could meaningfully affect the calculated average rates of voltage change. A more robust method could have been implemented to determine which data points represented stable charging behavior and which points were affected by transitions or noise. Additionally, because the interval boundaries were estimated visually from the graphs, some relevant data points may not have been fully considered in our

analysis. However, the selected intervals provided reasonable representations of the major charging stages visible in the data.

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.
```

```
time = cycleData.DateTime;
voltage = cycleData.Voltage;
current = cycleData.Current;
power = voltage .* current;
```

```
% STEP 1: Compute the rate of voltage change at different key time
% intervals. This helps you mathematically characterize the battery's
% charging profile and is more reliable than simply eyeballing the voltage
% vs. time plot.
% Optional: plot the points of notable transitions in charging dynamics in
% the voltage vs. time plot generated above
% Hint: explore using the gradient function
```

```
chargeTimeMask = time >= 0 & time <= 1850; % logical mask for charge time
values
chargeTime = time(chargeTimeMask); % time vector containing indices where
chargeTimeMask is true
chargeVoltage = voltage(chargeTimeMask);
chargeCurrent = current(chargeTimeMask);
chargePower = power(chargeTimeMask);
voltageChange = gradient(chargeVoltage, chargeTime); % vector of
derivatives at every point of the voltage vs. time graph
```

```
% For each time interval (0 <= t <= 300, 300 <= time <= 600, 600 <= time <=
% 1250, 1250 <= time <= 1850) the first and last 50 seconds were cut for
% intervals 1, 2, and 4 to reduce noise at the edges of the data while 100
% seconds were cut for interval 3 because of additional noise. The intervals
% were estimated from the graphs in Task 2.
```

```
interval1 = chargeTime >= 50 & chargeTime <= 250;
interval2 = chargeTime >= 350 & chargeTime <= 550;
interval3 = chargeTime >= 700 & chargeTime <= 1150;
interval4 = chargeTime >= 1300 & chargeTime <= 1800;
```

```
% Within these intervals, the average was calculated to give an estimate of
% the rate of voltage change at each of the intervals
```

```
rateOfChange1 = mean(voltageChange(interval1));
rateOfChange2 = mean(voltageChange(interval2));
rateOfChange3 = mean(voltageChange(interval3));
rateOfChange4 = mean(voltageChange(interval4));
```

```
% STEP 2: Compute the time required to reach 80% charge and 100% charge
```

```

chargeTime100 = chargeTime(end) - chargeTime(1); % find the overall charge
time
totalCharge = cumtrapz(chargeTime, chargeCurrent); % using cumtrapz
function to get a vector containing
% the total charge at every point by integrating the current vs. time graph
targetCharge80 = 0.8 * totalCharge(end);
index80 = find(totalCharge >= targetCharge80, 1); % finds first index where
chargeTime80 = chargeTime(index80) - chargeTime(1);

% STEP 3: Use integration to calculate the area under the power-time curve.
% This represents the total energy delivered to the battery during charging.
% Hint: explore using the trapz function

energyDelivered = trapz(chargeTime, chargePower);

% STEP 4: Estimate the resistive energy loss.
% Hint: use Joule's heating formula ( $P = I^2 * R$ ) to estimate power loss in
% watts. The energy loss is given as  $E = P * t$ , where t is time in seconds
% and energy is in joules.

internalResistance = 0.0176; % The listed internal resistance was assumed
to be in ohms
powerLoss = chargeCurrent.^2 * internalResistance;
energyLoss = trapz(chargeTime, powerLoss);

% STEP 5: Save results to a summary table. Display the table.

quantity = categorical(["Average voltage rate: 50-250s (V/s)";
    "Average voltage rate: 350-550s (V/s)";
    "Average voltage rate: 700-1150s (V/s)";
    "Average voltage rate: 1300-1800s (V/s)";
    "Time to reach 80% charge (s)";
    "Time to reach 100% charge (s)";
    "Energy delivered during charging (J)";
    "Resistive energy loss during charging (J)"]);

value = [rateOfChange1;
    rateOfChange2;
    rateOfChange3;
    rateOfChange4;
    chargeTime80;
    chargeTime100;
    energyDelivered;
    energyLoss];

summaryTable = table(quantity, value, 'VariableNames', {'Quantity',
    'Value'});
disp(summaryTable);

```

Quantity	Value
Average voltage rate: 50–250s (V/s)	0.00056501
Average voltage rate: 350–550s (V/s)	0.00018866
Average voltage rate: 700–1150s (V/s)	0.00018956
Average voltage rate: 1300–1800s (V/s)	–1.0765e–06
Time to reach 80% charge (s)	596.6
Time to reach 100% charge (s)	1846.7
Energy delivered during charging (J)	13679
Resistive energy loss during charging (J)	326.37

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning (for example, why would engineers want to compute the analytical results in Task 3?):

In Task 3, the energy delivered, resistive energy loss, time to reach 80% charge, and the time to reach 100% charge, and the average rate of voltage change during key charging intervals were calculated. Engineers would want to compute these analytical results because they provide information about the charging performance and energy efficiency of the battery. For example, if most of the electrical energy is lost to internal resistance, the battery isn't as effective because most of the energy is being converted into unwanted heat rather than usable power. Through continuous analysis of a battery's charging profile, engineers can continue to make changes to their project in order to minimize energy loss.

What are some limitations of your work?

Some limitations of this work are that we are only analyzing one charging cycle and assuming that the battery's internal resistance remains constant throughout the charging process. The times corresponding to 80% and 100% charge are also based on the recorded charging cycle, rather than a direct measurement of the battery's true state of charge. Additionally, the key charging intervals were estimated visually from the graphs, so changing the selected intervals could slightly change the calculated average rate of voltage change. Other factors such as temperature, battery age, and additional sources of energy loss were not considered.

What are practical next steps?

Some practical next steps would be to analyze multiple charging cycles to determine whether the results are consistent and to study how charging behavior changes under different conditions. Future analysis could also include temperature measurements, changes in internal resistance over time, and a more direct measurement of state of charge. The process for identifying charging intervals could also be automated rather than estimating the transition visually. These improvements would provide a more complete picture of the battery's charging efficiency and performance.